Build Your Own AI Bot — Trainer's Course Plan

For students with little or no technical background

Total: 16 hours taught + 6 hours practice = 22 hours Outcome: every student walks out with a working bot that answers questions from their own documents

---

PART 0 — COURSE DESIGN

0.1 Who this is for

People who:

- Use ChatGPT or similar, but have never built anything with AI
- Cannot write code, or can only read a little
- Work with documents — policies, manuals, reports, contracts, tickets
- Need to explain or own an AI project at work, not necessarily engineer it

0.2 What they will be able to do at the end

```
✓ Explain, in plain words, how an AI bot answers from company documents
✓ Build a working bot from their own PDFs and documents — no coding
✓ Diagnose why a bot gives a wrong answer, and fix it
✓ Know which settings to change, and what each one does
✓ Estimate what it costs to run
✓ Know when to ask an engineer for help, and what to ask for
```

0.3 What they will NOT be able to do — say this on day one

> "You will not be a machine learning engineer after this course. You will not be writing production software.
>
> What you will be is the person in the room who **understands what is actually happening**, can build a working prototype yourself, and can tell the difference between a

> vendor who knows what they're talking about and one who doesn't.
>
> In most organisations, that person is rarer and more useful than the engineer."

Setting this boundary early prevents disappointment on day four.

0.4 Timeline options

Pick one based on your audience's availability.

Option A — Four half-days (recommended)

Day	Hours	Focus	End state
Day 1	4h	Understanding: how AI works, why it forgets, what RAG is	Bot answering from 1 document
Day 2	4h	Building the library: documents in, cut up, filed	Bot answering from their own 20 documents
Day 3	4h	Finding the right page: search, ranking	Noticeably better answers, tuned by them
Day 4	4h	Answering well: prompts, citations, testing, cost, launch	Finished bot + a test sheet + a cost estimate

Between days: 1.5 hours homework each. That's the 6 hours of practice.

Option B — Two full days (intensive)

Same content, 8 hours per day. **Warning:** attention collapses after hour 5. Only do this if travel makes four days impossible. Add extra breaks and cut Module 11.

Option C — Eight evening sessions (2 hours each)

Best retention, worst momentum. Students forget the tool between sessions. If you use this, start each session with a 10-minute "where we got to" rebuild.

Option D — Add Day 5 (4h) — the advanced day

Only for students who finished Day 4 comfortably. Covers Advanced, Hybrid and Agentic RAG. Details in Part 6.

0.5 Tools — this decision matters more than anything else

For a non-technical audience, **the tool must let them see and change each layer without code.** Otherwise the concepts stay abstract.

The two-tool approach

Tool 1 — for the Hour One Win (zero setup)

Option	Why
Google NotebookLM	Upload PDFs, ask questions immediately. Free. No account setup friction. Best for "look, this works" in five minutes.
Claude Projects / ChatGPT Custom GPT	Upload documents to a project, ask questions. Also instant.

Use this in the first 30 minutes so students experience the outcome before they learn the machinery. **Never start with theory.**

Tool 2 — for the real build (visible layers)

Option	Why it fits non-coders	Note
Dify	Open source, browser-based, drag-and-drop. Each RAG layer is a visible block you can click and change. **Best default.**	Self-host or use their cloud
Flowise	Drag-and-drop boxes wired together. Very visual — the pipeline *looks* like the diagram you drew.	Slightly more fiddly setup
LangFlow	Same idea as Flowise	Similar
n8n	Automation-first; good if they also want to connect email, Slack, etc.	More general-purpose
Botpress / Chatbase	Commercial, polished, fastest to a shippable bot	Fewer visible knobs — less teaching value

> **Trainer recommendation: NotebookLM for hour one, Dify for the build.** Dify shows chunk size, retrieval method, top-k, reranking and the prompt as editable settings — which means every concept you teach has a visible control the student can move.

An optional code track for the one or two students who want it: a Python notebook in Google Colab using LangChain. Offer it as a take-home, never as the main path. Mixing tracks loses the room.

0.6 Room setup checklist

- Every student has a laptop with a browser. Tablets will not work.
- Accounts created **before day one** — chasing signups burns 40 minutes
- Wi-Fi tested with 20 simultaneous connections
- Your Dify instance running and tested with a sample document
- A shared folder of sample documents — so students without their own can still build
- One messy real PDF (two columns, a table, ideally a scan) for the parsing demo
- Projector showing your screen; students follow along on theirs
- A printed one-page glossary per student (Part 5)

> **The single biggest cause of a failed hands-on course is setup time.** Do accounts and installs by email a week before. Have a fallback shared login for anyone who couldn't.

PART 1 — THE MASTER ANALOGY

Use one analogy for the entire course. Do not switch. Everything maps to it.

1.1 The company library

Introduce it in the first 20 minutes and return to it in every single module.

> "Imagine your company hires a brilliant new assistant.
>
> This person has read enormous amounts — every textbook, every public website, most published books. They are genuinely knowledgeable and they write beautifully.
>
> **But they have never worked here.** They have never seen your policies, your contracts, your product manuals, your customer history. Not one page.
>
> Now — if you ask them 'what is our refund policy?', what happens?

> They don't say 'I don't know.' They give you an answer that *sounds* completely right. Confident, well-written, plausible. And entirely invented, because they had nothing to go on.
>
> **That is exactly what an AI model does. And that made-up answer has a name — a hallucination.**
>
> So what would you actually do with a new assistant? You wouldn't fire them. **You'd build them a library.**
>
> You'd gather the documents. Organise them. Set up a filing system. And then, when someone asks a question, you'd pull the right pages and put them on the desk *before* the assistant answers.
>
> **That's it. That's the whole course.** Everything we do for the next four days is building that library and teaching the assistant to use it.
>
> The technical name for it is RAG — Retrieval Augmented Generation. Ignore the name. **It means: fetch the right pages first, then answer.**"

1.2 The mapping — put this on a wall poster

This is the spine of the entire course. Every module is one row.

In the library	In the bot	Module
Papers scattered around the company	Your documents and data sources	4
Photocopying them properly	Reading the files (parsing)	5
Throwing out duplicates and junk pages	Cleaning	5
Cutting long books into index cards	Chunking	6
Writing the source on each card	Metadata	6
Marking which cards are confidential	Permissions	6
Filing cards by topic, not alphabetically	Embeddings	7
The filing cabinet itself	Vector database	7
Understanding what the person actually wants	Query understanding	8
Fetching 50 cards that might help	Retrieval	8

Also checking the exact title index	Keyword search / hybrid	9
Reading those 50 and picking the best 5	Reranking	9
Laying the 5 cards neatly on the desk	Context assembly	9
The assistant reading them and writing an answer	Generation	10
Someone checking the answer matches the cards	Verification	10
A quiz with known correct answers	Evaluation	11
Counting what the whole thing costs to run	Cost	12
A librarian who goes back to the shelves repeatedly	Agentic RAG	Day 5

> **Trainer note:** print this table as an A3 poster and stick it on the wall. Point at the current row at the start of every module. Students who get lost can find themselves on it.

PART 2 — THE MODULES

DAY 1 — UNDERSTANDING (4 hours)

Goal: they understand why this technology behaves the way it does, and they've seen a bot work.

Module 1 — The Hour One Win (30 min)

Start with the outcome. Theory comes after.

What you do

```
0–5 min     Everyone opens NotebookLM (or your chosen instant tool)
```

```
5–10 min    Everyone uploads 2–3 PDFs — theirs if they have them,
            yours from the shared folder if not
10–20 min   Everyone asks 5 questions of their own documents
20–30 min   Discussion
```

The discussion — the important part

Ask these three questions in order:

1. **"What worked well?"** — let them be impressed
2. **"What got it wrong, or where did it struggle?"** — this is the seed of Days 2–4
3. **"Where did it get the answer from? Can you check it?"**

> "Hold on to the things that went wrong. **Every one of them is a lesson in this course.** By Thursday you'll know exactly which part of the machine caused each of them, and how to fix it."

Write the failures on a flipchart. Return to that flipchart on Day 4. It's the most satisfying moment of the whole course.

Module 2 — How the AI actually works (60 min)

Goal: kill the "there's a database inside it" belief. Everything else depends on this.

2.1 It's a prediction machine (15 min)

> "You already know what this is. It's the autocomplete on your phone.
>
> When you type 'I'll be there in five ___', your phone suggests 'minutes'. It's not thinking. It's noticed that after those words, 'minutes' usually comes next.
>
> Your phone is mediocre at this because it's only seen your messages.
>
> **Now imagine the same thing, but it has read most of the internet.** The suggestions stop being mediocre and start being genuinely useful.
>
> That's it. That's the entire mechanism. **There is no second trick.**"

2.2 Why it makes things up (15 min)

> "Here's the consequence, and it explains almost every problem you'll hit.
>
> **An autocomplete never stops.** It always has a next word. That's what it was built to do.

> So when it doesn't know something, it doesn't say so. It produces the most plausible-sounding continuation anyway — fluent, confident, and wrong.
>
> **That's a hallucination. It isn't a fault. It's the machine doing exactly its job, in a situation where its job is the wrong thing to do.**"

Live demo (5 min): ask a chatbot about something obscure and specific from your own company. Watch it produce a confident, wrong answer. This lands harder than any explanation.

2.3 The closed-book exam (15 min)

Introduce the master analogy properly here. Use the full script from Part 1.1.

⚠️ **The misconception to kill explicitly:**

> "The most common wrong belief in this room right now is that the AI has a database inside it — that facts are stored somewhere and looked up, like Google.
>
> **There is no database. Nothing is stored. Nothing is looked up.**
>
> Think of a person who has read ten thousand books and is now in an exam hall with no books. They remember ideas brilliantly. Ask them the exact figure from page 247 of one textbook and they'll give you *a* number, confidently. It may be wrong.
>
> **This is the entire reason the rest of this course exists.**"

2.4 Q&A (15 min)

Let them ask anything. This is where the real misconceptions surface. Common ones and short answers are in Part 5.

Module 3 — What RAG is, and what it isn't (60 min)

3.1 The five steps (20 min)

Draw this on the whiteboard. Do not use a slide — draw it, slowly, while talking.

```
SETTING UP THE LIBRARY  (done once, then updated)
─────────────────────────────────────────────────

  1.  Gather your documents
  2.  Cut them into index cards
  3.  File the cards by topic

ANSWERING A QUESTION  (happens every time someone asks)
```

```
4.  Understand the question, fetch the best cards
5.  Give those cards to the assistant, who reads them and answers
```

> "Five steps. That's the whole system. Everything for the next three days is making each of these five steps better."

3.2 Why not just give it everything? (20 min)

Students always ask this. Answer it before they do.

> "Fair question. The AI can hold about 1,500 pages at once now. So why not just paste the whole handbook in every time?
>
> **Four reasons.**
>
> **One — money.** You pay by the page, every single time someone asks. Not once. Every time.
>
> **Two — speed.** More pages in, longer wait for the answer.
>
> **Three — and this surprises people — accuracy gets worse.** If I hand you the entire company archive and ask for the refund policy, you'll find it eventually, but you might grab an old version. If I hand you the one right page, you answer immediately and correctly.
>
> **Four — permissions.** You cannot paste everything in and still control who sees what. HR documents would reach everyone."

The number — put it on a slide, it's the one figure from the course they'll repeat to their boss:

```
A 500-page handbook. 1,000 questions a day.

  Paste everything in every time:   about  $20,000 per month
  Fetch just the right 8 pages:     about     $300 per month

  Roughly 66 times cheaper.
  And the answers are better, because the assistant isn't
  wading through 500 pages to find one paragraph.
```

3.3 What RAG cannot do (20 min)

Teach the limits early. Students who know the limits build better things and ask better questions.

> "Six things RAG is bad at. Know these now, not after you've built something.
>
> **1. It cannot count.** Ask 'how many refunds last quarter' and it fetches five cards mentioning refunds, then produces a total that is confidently wrong. Counting needs a spreadsheet or database, not a library.
>
> **2. It cannot summarise a whole book.** Ask 'summarise this 200-page report' and it fetches five cards. That's five paragraphs, not a summary.
>
> **3. It struggles with questions needing several steps.** 'Which contracts are affected by the policy that covers product X, and when do they renew?' — that's three lookups. Basic RAG does one.
>
> **4. It's weak on exact codes and 'not'.** 'Contract was terminated' and 'contract was NOT terminated' look almost identical to the filing system. We'll fix this on Day 3.
>
> **5. If the right card isn't fetched, nothing else can save you.** No amount of clever wording fixes a card that was never pulled off the shelf.
>
> **6. Old cards survive.** Delete the document, and unless you're careful, the card stays in the cabinet. Your bot then quotes a policy that no longer exists."

> **Say this:** "I'm telling you the weaknesses on day one because anyone who only tells you the strengths is selling you something."

Module 4 — Your documents: what goes in the library (60 min)

4.1 Choosing your corpus (20 min)

> "Before anything technical — **what should be in the library?**
>
> This is a judgement call, not a technical one, and getting it wrong wastes everything downstream."

The four questions to ask of any document:

```
1.  Is it CORRECT?      Wrong documents produce wrong answers, confidently.
2.  Is it CURRENT?      Old policies are worse than no policies.
3.  Is it APPROVED?     Should this be an official source of truth?
4.  Who may SEE it?     If not everyone, note it now — not later.
```

⚠️ The rule that saves them months:

> "**Do not put everything in.** The instinct is to dump every folder in the company and let

> the AI sort it out.
>
> It won't. It will find the 2019 draft policy alongside the 2026 approved one, and it has no way to tell which you meant.
>
> **A small, clean, trusted library beats a huge messy one. Every time.**
>
> Start with 20 documents you'd be happy to stand behind. Not 2,000."

4.2 Document types and what to expect (20 min)

What you have	How well it works	What to watch
Normal Word docs, clean PDFs	Very well	Nothing much
PDFs with two columns	Badly by default	Text gets read across the columns and scrambled
Scanned documents / photos	Depends entirely on scan quality	Needs extra processing; check a sample by eye
Spreadsheets	Depends on the question	Lookups work. **Totals and counts do not.**
PowerPoint	Reasonably	The speaker notes are often the real content
Web pages	Well, once cleaned	Menus and cookie banners become junk cards
Emails / chat	Messy	Very noisy; privacy concerns
Live data (order status, stock)	**Don't put this in the library**	It changes; connect to it directly instead

4.3 HANDS-ON LAB 1 (20 min)

```
Task:  Each student chooses 10–20 documents for their bot.

Steps:
  1.  Pick a real use case — "answer HR policy questions",
      "help with our product manual", "onboarding assistant"
  2.  Collect 10–20 documents for it
  3.  For each, note: is it current? is it approved? who may see it?
```

```
   4.  Discard anything that fails those checks
   5.  Put the survivors in one folder

Deliverable: a clean folder, and one sentence describing
             what their bot is for.
```

> **Trainer note:** walk the room during this. The most common mistake is picking a use case that's too broad — "answer any company question." Push them to narrow it. "HR leave policy questions" is a good first bot. "Everything" is not.

Day 1 homework (1.5 hours)

```
1.  Finish your document folder — 10–20 files, cleaned up
2.  Write down 10 real questions your bot should answer
    (real ones people actually ask, not invented ones)
3.  For 3 of those questions, write the correct answer yourself
    → you'll need these on Day 4 for testing
```

DAY 2 — BUILDING THE LIBRARY (4 hours)

Goal: their own documents go in, get cut up properly, and get filed. By the end they have a working bot on their own content.

Module 5 — Reading the documents properly (60 min)

Library step: photocopying the papers.

5.1 Why this is harder than it sounds (20 min)

> "Step one of building the library is getting the words out of your files. Sounds trivial. It is not — and **this step causes more bad answers than anything else in the whole system.**"

Live demo — do this, it's the most memorable moment of Day 2:

Take your messy two-column PDF. Run it through a basic text extractor. Show the output on the projector.

> "Look at what happened. This document has two columns. The extractor read straight

> across the page — so it took the first line of the left column, then the first line of the right column, and glued them together.
>
> The result is nonsense. Grammatically it looks like text. But the sentences are scrambled.
>
> **And here's the dangerous part: nothing anywhere reports an error.** Your bot will happily file this nonsense, retrieve it, and answer from it. Confidently."

Then show the same page through a layout-aware tool. Show the difference.

5.2 Tables and pictures (20 min)

> "Two more things that break.
>
> **Tables.** A table is meaning-by-position — the number sits under a column heading and beside a row label. Flatten that into a line of text and you get '2024 2025 Revenue 450 520 Costs 300 310'. Meaningless.
>
> **Pictures, charts, and diagrams.** A basic extractor pulls out the words and ignores everything else. If your answer is in a flowchart, it's simply gone.
>
> For both, you need tools that understand layout — or, for the important ones, have an AI describe the picture in words and file *that* description."

5.3 What to do about it (10 min)

```
Rule 1   Open 3 of your documents and look at the extracted text
         BY EYE before you build anything. Five minutes. Saves weeks.

Rule 2   If you have two-column PDFs, scans, or important tables,
         say so when you set up the tool — most let you pick a
         better reading method.

Rule 3   If a document extracts badly and it matters, fix the
         document. Re-export it. It's faster than fighting the tool.
```

5.4 Cleaning up (10 min)

> "Two more jobs before filing.
>
> **Remove the repeated junk.** Page headers, footers, 'Confidential — Internal Use Only' on every page. If you leave them, they appear on every single card, and they match every question. They're noise that crowds out your real content.
>
> **Remove duplicates.** Every organisation has `policy_final_FINAL_v2.docx`. If three near-

> identical versions are in the library, your bot fetches five cards and three are the same thing — possibly from the wrong version."

Module 6 — Cutting documents into cards (75 min)

Library step: cutting books into index cards. This is where most quality is won or lost.

6.1 Why we cut things up at all (15 min)

> "Why not file whole documents?
>
> Two reasons.
>
> **Too big to hand over.** A 200-page manual won't fit on the desk, and even if it did, you're paying for 200 pages when the answer was on one.
>
> **Too vague to find.** If you file a whole manual under one topic, it's filed under *everything*. It matches every question and answers none of them well.
>
> So we cut it into cards. **Each card should be one coherent idea.**"

6.2 The Goldilocks problem (20 min)

Draw this on the whiteboard.

```
CARDS TOO SMALL                CARDS TOO BIG
_______________                _____________

"Notice must be given          The entire 40-page
 in writing."                   HR chapter

Problem: notice of what?        Problem: mostly irrelevant.
By whom? To whom? How           You pay for 40 pages to
many days? The card is          get one sentence, and the
useless on its own.             important bit gets buried.
```

> "So the card must be **big enough to stand on its own, small enough to be about one thing.**
>
> A good rule: **a card should be about one page of text.** Roughly 300 to 500 words. If you read it cold, you should be able to tell what it's about."

6.3 Where to cut (20 min)

> "Now the important part — **where** do you cut?

> **The bad way — cut every 500 words.** Chop, chop, chop, regardless of what's there. This cuts sentences in half. It splits tables from their headings. It separates a rule from its exception.
>
> **The good way — cut where the document already has breaks.** At headings. At section boundaries. At the end of a topic. The author already decided where one idea ends and the next begins. **Use their decision.**"

Three rules to write down:

```
Rule 1   Cut at headings and sections, not at word counts.

Rule 2   Overlap slightly — repeat the last sentence or two of the
         previous card at the top of the next one. So a sentence
         that happens to sit on a boundary isn't lost.

Rule 3   NEVER split a table. If a table is too big, repeat the
         header row on every piece.
```

6.4 The single best trick in this course (20 min)

This is the highest-value ten minutes of Day 2. Slow down.

> "Here is a trick that costs almost nothing and improves your bot more than anything else you'll do.
>
> Read this card, out loud, and tell me what it's about:
>
> *'The amount shall be recognised evenly over the period, unless the conditions in clause 3 are not met.'*
>
> No idea, right? Which amount? What period? What is this policy even about?
>
> **The problem: the context that made this meaningful was left behind in the document.** The chapter title, the section heading, the topic — all of it got cut away when we made the card.
>
> **The fix: write one sentence at the top of every card saying where it came from.**
>
> *'This is from the FY25 Revenue Recognition Policy, section 4, about deferred revenue. The amount shall be recognised evenly over the period, unless...'*
>
> Now it's findable. Now, when someone asks about deferred revenue, this card surfaces.
>
> Most tools can add this automatically. **Turn it on.** It is the cheapest large improvement available to you."

6.5 Labelling the cards (15 min)

> "Last job before filing. Write on each card:
>
> - **Which document it came from** — so the bot can cite it, and a human can check
> - **Which page or section** — so they can find it in the original
> - **How old it is** — so you can prefer newer over older
> - **Who's allowed to see it** — and this one matters enormously
>
> **On that last point.** If your library has documents not everyone should read — salaries, performance reviews, unannounced projects — you must mark them **now**, as you file them.
>
> Adding permissions later is genuinely painful and usually incomplete. And the failure mode is severe: a helpful bot cheerfully telling anyone who asks what the CEO earns.
>
> **If in doubt: don't put it in the library at all.**"

Module 7 — Filing the cards (75 min)

Library step: the filing system and the cabinet.

7.1 Filing by meaning, not by alphabet (25 min)

> "Now, how do we file these cards so we can find them?
>
> **Alphabetically won't work.** Someone asks 'how do I get my money back?' — there's no card starting with 'how do I get'. The card says 'Refund Policy'. Alphabetical filing finds nothing.
>
> We need to file by **meaning**. And here's how computers do that, and it's genuinely clever."

The colour explanation — use this, it works better than anything else:

> "You already understand a system exactly like this. It's how colours work on a screen.
>
> Every colour is three numbers — how much red, green, blue.
>
> ```
> Crimson     =  [220,  20,  60]
> Firebrick   =  [178,  34,  34]
> Sky blue    =  [135, 206, 235]
> ```

> **Now notice something remarkable. You can tell which two are similar without looking at them.** Crimson and firebrick have close numbers. Sky blue is far off on all three.
>
> **Similarity became arithmetic.** Nobody wrote a rule saying 'crimson resembles firebrick'. It falls out of the numbers.
>
> The filing system does exactly this for meaning — except with about 1,500 numbers instead of three.
>
> ```
> "refund policy"           →  a list of ~1,500 numbers
> "how to get money back"   →  a very similar list
> "quarterly revenue"       →  a completely different list
> ```
>
> The first two share **no words at all**. But their numbers are close. So when someone searches for one, the system finds the other.
>
> **That's it. That's how a bot finds the right page when you use different words than the document does.**"

7.2 The filing cabinet (15 min)

> "Where do these number-lists live? In something called a vector database. Ignore the name — **it's a filing cabinet built to answer one question very fast: 'which cards have numbers closest to this?'**
>
> You don't need to choose one on day one. Whatever tool you're using has one built in. When your engineers ask which one, the honest answer for most companies is: **if you already run a normal database, use that. Don't buy a new system until you've outgrown the one you have.**"

7.3 The weakness you must know (15 min)

> "This filing system has one specific blind spot, and it causes real problems.
>
> ```
> "The contract was terminated"
> "The contract was NOT terminated"
> ```
>
> To the filing system, these are almost identical. Ninety-six percent of the words are the same. The one word that reverses the meaning barely moves the numbers.
>
> Back to colours — the numbers tell you two colours look alike. They cannot tell you one is a warning light and the other is an all-clear.

> **The blind spots are: 'not', numbers, dates, part numbers, and error codes.**
>
> We fix this tomorrow. It's the first thing we do on Day 3."

7.4 HANDS-ON LAB 2 (20 min)

```
Task:  Build your library and get a working bot.

Steps in Dify (or your tool):
  1.  Create a new Knowledge Base
  2.  Upload your 10-20 documents
  3.  Set chunk size to about 500 words
  4.  Turn ON the option to add context to each chunk
      (may be called "contextual", "parent-child", or "Q&A mode")
  5.  Let it process
  6.  LOOK AT THE CHUNKS. Most tools show you them. Read five.
      Do they make sense on their own?
  7.  Create a chat app, connect it to your Knowledge Base
  8.  Ask your 10 questions from homework

Deliverable: a bot answering from their own documents,
             and notes on which of the 10 questions it got wrong.
```

> **Trainer note:** step 6 is the one they'll skip. Make them do it. Reading five actual chunks teaches more about chunking than any explanation you can give.

Day 2 homework (1.5 hours)

```
1.  Ask your bot all 10 questions. Write down each answer.
2.  Mark each: GOOD / PARTLY RIGHT / WRONG / MADE IT UP
3.  For every wrong one, find the chunks it used
    (your tool will show you) and write down: was the right
    information even in there?
4.  Bring your list tomorrow.
```

> **This homework is the heart of the course.** It teaches the single most important diagnostic habit: **look at what was retrieved before blaming the AI.**

DAY 3 — FINDING THE RIGHT PAGE (4 hours)

Goal: their bot's answers get measurably better, through changes they make

themselves.

Module 8 — Understanding the question (60 min)

8.1 The homework debrief (20 min)

Go round the room. For each wrong answer, ask one question:

> **"Was the correct information in the chunks it fetched?"**

Tally the answers on the whiteboard:

```
Right info was NOT fetched      →  ___ cases   (a FINDING problem)
Right info WAS fetched, but
the answer was still wrong      →  ___ cases   (an ANSWERING problem)
```

In almost every room, the first number is much bigger.

> "Look at that. **Most bad answers aren't the AI being stupid. They're the right page never reaching the desk.**
>
> That's the most valuable thing you'll learn this week. When your bot is wrong, **don't rewrite the instructions. Look at what it fetched.**"

8.2 The question isn't the search (25 min)

> "Here's something that surprises people. **The question the user types is rarely the best thing to search with.**
>
> Three examples.
>
> **One — follow-up questions.** Someone asks 'what's our leave policy?' then 'what about for contractors?'
>
> That second question, on its own, is meaningless. Search for 'what about for contractors' and you get nothing useful. The system has to rewrite it first: *'what is the leave policy for contractors?'*
>
> **If your bot is bad at follow-up questions, this is almost always why.** It's the single most common fault in chat bots.
>
> **Two — questions don't look like documents.** People ask short questions. Documents contain long formal statements. 'Can I get a refund?' doesn't look much like 'Section 7.2: Customer Reimbursement Provisions.' Some systems generate a *fake answer* first and

> search with that, because fake answers look more like documents than questions do.
>
> **Three — hidden filters.** Someone asks 'what did Legal update after January?' Buried in there are two filters: department is Legal, date is after January. A good system pulls those out and uses them to narrow the search before it starts."

8.3 Not every question needs a search (15 min)

> "One more. 'Hello', 'thanks', 'can you say that more simply?' — none of these need a trip to the library.
>
> Searching anyway costs money and adds delay. Good systems check first: **does this question need the library at all?**"

Module 9 — Two ways to search, and picking the best (75 min)

9.1 Fixing yesterday's blind spot (25 min)

> "Yesterday I told you the filing system is bad at exact things — error codes, part numbers, 'not'. Here's the fix, and it's simple.
>
> **Use two search methods at once.**
>
> **Method 1 — search by meaning.** What we built yesterday. Finds 'refund policy' when you ask 'how do I get my money back'. Brilliant at understanding. **Bad at exact strings.**
>
> **Method 2 — search by exact words.** Essentially a very good Ctrl+F. Finds 'ERR_SVC_4471' exactly. **Useless with synonyms** — it will never connect 'refund' to 'money back'.
>
> Neither is enough alone. **And your users ask both kinds of question**, often in the same conversation. 'My payment is stuck' is a meaning question. 'What does error 4471 mean' is an exact question.
>
> So run both, and combine the results. The industry name is **hybrid search**. In most tools it's a dropdown. **Change it.**"

9.2 How the two lists get combined (15 min)

Do this on the whiteboard with real numbers. It demystifies something that sounds complicated.

```
Someone asks about "refund for damaged goods"
```

```
MEANING SEARCH says:        EXACT SEARCH says:
  1st   Card A                1st   Card C
  2nd   Card B                2nd   Card A
  3rd   Card C                3rd   Card D

Now — which cards should win?

Card A:  1st in one list, 2nd in the other   → appears high in BOTH  ★
Card C:  3rd in one list, 1st in the other   → appears high in BOTH  ★
Card B:  2nd in one list, absent from other  → only one list
Card D:  absent, 3rd                          → only one list

FINAL ORDER:  A,  C,  B,  D
```

> "The logic is common sense: **if two different methods both think a card is relevant, it probably is.** Agreement is evidence.
>
> The system does this with a small formula, but that's all the formula is saying."

9.3 The shortlist trick (20 min)

This is the biggest single improvement available. Give it proper time.

> "Now, the most valuable setting in your entire bot. And you already understand it, because you've done it.
>
> **How does hiring work?**
>
> Five hundred applications arrive. A filter reduces them to fifty — fast, cheap, a bit crude. Then a human reads those fifty properly and picks five. Slow, expensive, accurate.
>
> **You wouldn't do either step alone.** The filter alone gives you fifty mediocre candidates. A human alone can't read five hundred.
>
> **Search works exactly the same way.**
>
> The filing cabinet is the fast crude filter — brilliant at reducing thousands of cards to fifty. Then a second, smarter step reads those fifty properly and picks the best five.
>
> That second step is called **reranking**. In your tool it's usually a toggle labelled 'rerank' or 'rerank model'.
>
> **Turn it on. It's typically a ten to twenty-five percent improvement in answer quality, and it costs a fraction of a cent per question.**
>
> If someone tells you their bot is 'about seventy percent right', this is the first thing to try. Every time."

9.4 How many cards to fetch (15 min)

> "One number you'll be asked to set: how many cards to hand over. Usually called 'top K'.
>
> ```
> Too few  (1–2)   You miss the answer when it's spread across pages
> Too many (20+)   The important card gets buried; you pay more;
>                  accuracy actually DROPS
> ```
>
> **With reranking on: fetch 50, keep the best 5. Without reranking: fetch 5 to 8 and stop.**
>
> And remember Day 1: **five right pages beat fifty mediocre ones.** More is not better."

Module 10 — HANDS-ON LAB 3: make it better (45 min)

```
Task:  Improve your bot using only settings. Measure the difference.

BEFORE YOU CHANGE ANYTHING:
  Ask your 10 questions. Score each 1–5. Write down the total.
  ← this is your "before" number

NOW CHANGE ONE THING AT A TIME. Re-test after each.

  Change 1:  Switch search to HYBRID          → new score: ___
  Change 2:  Turn ON reranking                → new score: ___
  Change 3:  Set fetch=50, keep=5             → new score: ___
  Change 4:  Try chunk size 300, then 800     → new score: ___

RULE: one change at a time. If you change three things and it
      improves, you don't know which one helped — and next time
      you won't know what to change.
```

> **Trainer note:** this lab is where the course clicks. Students watch their own score go up because of a decision they made. Leave time for people to share their before/after numbers out loud — the room energises.

Day 3 homework (1.5 hours)

```
1.  Finish your tuning. Record your best settings.
2.  Write 10 MORE questions — this time include:
      - 2 that need information from two different documents
      - 2 that ask for a count or total
```

```
      - 2 that your library genuinely CANNOT answer
      - 1 with an exact code or number in it
3.  Test all 20. Note what happens to each type.
```

> The out-of-scope questions matter most. They set up Day 4's most important lesson: **a bot that says "I don't know" is better than one that guesses.**

DAY 4 — ANSWERING WELL, AND GOING LIVE (4 hours)

Goal: a finished, trustworthy, tested bot with a known running cost.

Module 11 — Instructing the assistant (60 min)

11.1 The most important sentence in your bot (20 min)

> "The cards are on the desk. Now we tell the assistant how to use them.
>
> **And there's one instruction that matters more than everything else combined.**"

Put this on screen and leave it there:

```
"Answer using ONLY the information in the documents provided.

 For every fact, say which document it came from.

 If the documents do not contain the answer, reply exactly:
 'I could not find this in the available documents.'

 Do not use your own knowledge. Do not guess."
```

> "Look at what each line does.
>
> **'Only the documents'** — stops it falling back on invented knowledge.
>
> **'Say which document'** — now a human can verify in seconds instead of trusting blindly.
>
> **'If not there, say so'** — and this is the clever one. **It turns a lie into a report.**
>
> Without it, when the library fails, you get a confident wrong answer and find out weeks

> later from a customer.
>
> With it, you get 'I could not find this'. **And now you can count how often that happens.** That number is a direct measure of how good your library is. If it rises, something broke — a document changed, something didn't get indexed.
>
> **You've turned an invisible problem into a number on a dashboard.**"

11.2 Tell it what the answer should look like (20 min)

> "Second rule: **describe the output, not the personality.**
>
> Writing 'you are a world-class expert' does very little on modern AI. What does a lot is being exact about the answer's shape.
>
> ❌ `'Summarise the policy.'`
>
> ✓
> ```
> 'Answer in at most three sentences.
>  State the number of days explicitly if one is mentioned.
>  End with the document name and section number.
>  If the policy differs by employee type, say so.'
> ```
>
> Notice the second one is *longer*. That's fine. **Clear beats short.**"

11.3 Things that don't work (10 min)

> "Save yourself time — these are internet folklore, not technique:
>
> - 'Please' and 'thank you'. Costs money at scale, changes nothing.
> - 'I'll tip you $200'. No.
> - 'You will be penalised if you're wrong'. No.
> - 'Be thorough but brief'. That's a contradiction. **Pick one and give a number.**
> - 'Don't be verbose' — weak. **'Answer in three sentences'** — strong. State the target, don't forbid the opposite."

11.4 The three-sentence rule for fixing bad answers (10 min)

> "When an answer is wrong, ask these in order. **In order.**
>
> **1. Was the right card fetched?** If not — it's a library problem. Go back to chunking and search. Rewriting instructions will not help.
>
> **2. Was the card fetched but ignored?** Strengthen the instruction, or reduce how many

> cards you send.
>
> **3. Was the card fetched, used, and the answer still wrong?** Now it's an instruction problem, or the card itself is wrong.
>
> **Most people start at step three. Almost every problem is at step one.**"

Module 12 — Testing: how you know it works (60 min)

12.1 Why you need a test sheet (15 min)

> "Here's the uncomfortable question I ask every team: **how do you know your bot got better?**
>
> Usually the answer is 'it feels better'. That isn't good enough, and here's why it fails.
>
> You change a setting. Three answers improve. **Two get worse — but you didn't re-check those two, so you don't know.** You ship it believing you improved things.
>
> **A bot without a test sheet isn't a system. It's a demo.**"

12.2 Building the test sheet (25 min)

> "It's a spreadsheet. That's all it is. Nothing technical."

Question	Correct answer	Type	Score	Notes
How many days annual leave?	20 working days (HR policy s.3)	simple		
Leave for contractors?	12 days, pro-rata	simple		
Difference between staff and contractors?	8 days fewer, no carry-over	multi-document		
What's the CEO's salary?	SHOULD SAY "not found"	out of scope		

The six question types every test sheet needs:

```
1.  Simple lookup        answer sits on one page
2.  Multi-document       needs two or more sources
3.  Comparison           "what's the difference between X and Y"
4.  Ambiguous            vaguely worded, like real users write
5.  Out of scope         ★ correct answer is "I don't know"
6.  Exact code/number    tests the exact-word search
```

> "**Type 5 is the one everybody forgets, and it's the most important.**
>
> A bot that answers 19 out of 20 correctly and confidently invents the twentieth is *more dangerous* than one that answers 15 and admits it doesn't know the other 5.
>
> Because with the first bot, **nobody knows which one was the lie.**"

12.3 Using it (20 min)

```
1.  30 questions is plenty to start. More is better, but 30 works.
2.  Use REAL questions — from emails, tickets, or asking colleagues.
    Invented questions are always cleaner than real ones, and they
    will flatter your bot.
3.  Score each 1–5. Add up the total.
4.  Re-run the WHOLE sheet after every change. Not just the ones
    you were trying to fix.
5.  Every time the bot fails in real use, add that question to the
    sheet. It's now a permanent test.
```

Module 13 — What it costs (45 min)

13.1 How you're charged (15 min)

> "You aren't charged per question, per document, or per user. **You're charged per word — sort of.**
>
> The unit is called a 'token'. It's roughly three-quarters of a word. Think of it like units on an electricity meter — you don't pay per light bulb, you pay per unit.
>
> ```
> 1,000 tokens  ≈  750 words  ≈  1.5 pages
> 1 page        ≈  700 tokens
> ```
>
> **And two things you need to know:**
>
> **One — what the AI writes costs about five times what you send it.** Reading is cheap.

> Writing is expensive.
>
> **Two — if you send the same thing repeatedly, most providers will discount it by about ninety percent.** So put the unchanging parts of your instructions at the *top*, and the changing parts at the bottom. That ordering alone saves serious money."

13.2 Working out your cost (20 min)

Do this on the whiteboard, slowly.

```
YOUR BOT, PER QUESTION:

  Instructions you always send       1,000 tokens
  The 5 cards you fetched            2,500 tokens
  The conversation so far            1,500 tokens
  The question itself                  100 tokens
  ──────────────────────────────────────────────
  Sent in                            5,100 tokens

  The answer written back              400 tokens

  ──────────────────────────────────────────────
  COST  (at typical mid-tier rates)

  Sending:  5,100 ÷ 1,000,000 × $2   = $0.0102
  Writing:    400 ÷ 1,000,000 × $10  = $0.0040
  ──────────────────────────────────────────────
  TOTAL PER QUESTION                 ≈ $0.014

  100 questions a day:
     $0.014 × 100 × 30 days           = $42 per month
```

> "Forty-two dollars a month. That's what most first bots actually cost. **Far less than people fear.**
>
> Now the two things that make it explode:
>
> **One — long conversations.** That 'conversation so far' line grows with every message. By message forty it might be 50,000 tokens instead of 1,500. **Your cost per message doubles and nothing on your dashboard tells you.** The fix: most tools have a setting to summarise or trim old messages. Turn it on.
>
> **Two — using the most expensive model for everything.** Most questions don't need the top model. **Use the mid-tier for normal question-answering** — the library did the hard work, the AI just needs to read and write."

13.3 The three settings that control cost (10 min)

```
1.  Which model      Mid-tier handles most question-answering fine
2.  Chat history     Turn on trimming/summarising of old messages
3.  Answer length    "Answer in three sentences" is cheaper than
                     letting it write an essay
```

Module 14 — HANDS-ON LAB 4: finish and launch (60 min)

```
Task:  Finish your bot. You leave with something you can show people.

  1.  Write your final instructions, including the grounding
      sentence from Module 11                          (10 min)
  2.  Build your test sheet — 20 questions minimum,
      including 3 out-of-scope ones                    (20 min)
  3.  Run the full sheet. Record your score.           (10 min)
  4.  Fix the worst failure. Re-run the sheet.         (10 min)
  5.  Work out your monthly cost using the Module 13
      template                                         (10 min)

Deliverable: working bot + test sheet + score + cost estimate
```

Module 15 — Closing (15 min)

15.1 Return to the Day 1 flipchart

Bring out the list of failures from Module 1. Go through them one by one.

> "This is what didn't work on Monday morning. Let's see where each one came from."

What went wrong on Day 1	What we now know it was
Missed something that was in the document	Cutting or reading problem — Day 2
Couldn't find a specific code or number	Needed exact-word search — Day 3
Bad answers on follow-up questions	Question wasn't rewritten — Day 3
Made something up	No grounding instruction — Day 4

Got a total wrong	RAG can't count — needs a database, Day 1
Used an out-of-date policy	Old document still in the library — Day 2

> "Every single thing that frustrated you on Monday has a name and a fix. **That's the course.**"

15.2 The five things to remember

> **1.** The AI has no database inside it. It's predicting the next word. That's why it invents things.
>
> **2.** RAG means fetching the right pages before answering. Five right pages beat fifty mediocre ones — cheaper *and* more accurate.
>
> **3.** When the bot is wrong, **look at what it fetched before you blame the AI.** Nine times out of ten the answer was never there.
>
> **4.** Turn on hybrid search and reranking. Two toggles. Biggest improvement available.
>
> **5.** A bot without a test sheet is a demo, not a system.

15.3 What to do on Monday

```
1.  Show your bot to three people. Watch what they ask.
    Real users ask things you'd never predict.
2.  Add every failure to your test sheet.
3.  Do NOT expand the library yet. Make 20 documents work
    properly before you add 200.
4.  Decide: is this useful enough to take further?
    If yes, you now know exactly what to ask an engineer for.
```

PART 3 — DAY 5 (OPTIONAL): ADVANCED, HYBRID, AGENTIC (4 hours)

Only run this for students who completed Day 4 comfortably.

Module 16 — The four levels (45 min)

> "Everything we built is called Basic RAG, plus a couple of upgrades. There are three levels above it. Here's the whole ladder, and — importantly — **when each one is the right answer.**"

Level	What it is	You need it when	Cost/speed
Basic	Fetch, answer	Simple FAQ over clean documents	Fastest, cheapest
Advanced	+ rewrite questions, hybrid search, reranking, better cutting	**Real company documents. Where most bots should be.**	Slightly slower, barely more expensive
Hybrid	+ several different libraries and a router deciding which to use	You have documents AND a database AND live systems	More complex, a bit slower
Agentic	The AI decides what to look up, checks its own work, tries again	Complex multi-step questions	**5–10× the cost, 10× slower**

⚠️ **The warning that matters most:**

> "The tempting mistake is to jump straight to Agentic because it sounds impressive.
>
> **An agent does not fix bad searching. It repeats it.**
>
> With a poor library, an agent fails five times instead of once — at five times the cost, taking ten times as long to fail.
>
> **You've already built Advanced. That's where most real systems should live. Go up a level only when you hit a wall you can name.**"

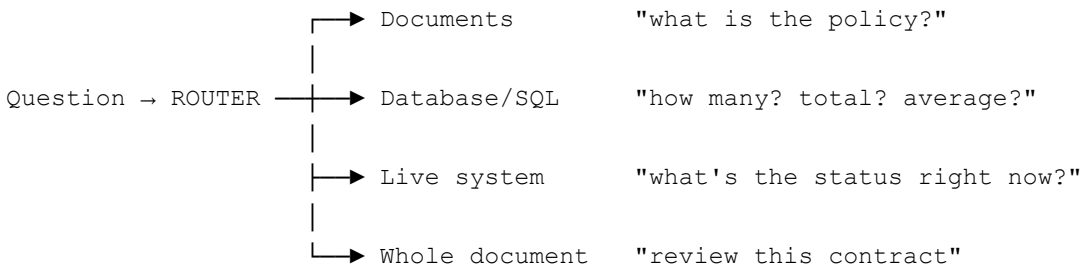
Module 17 — Hybrid: more than one library (60 min)

17.1 The question that breaks basic RAG (20 min)

> "Two questions. They look almost the same. **They need completely different machinery.**
>
> ```
> 'What is our refund policy?'
> 'How many refunds did we issue last quarter?'
> ```

> The first is a document question. The library finds the policy page.
>
> The second is a counting question. **The library cannot count.** It fetches five cards that mention refunds and produces a total that is confidently, invisibly wrong.
>
> Counting needs a spreadsheet or a database. **Different question, different tool.**"

17.2 The router (25 min)



> "The router is a small piece that looks at a question and decides where to send it.
>
> **It's the most valuable part of a hybrid system, and the one people most often skip.**
>
> And there's a bonus. **Log every routing decision.** The questions it gets wrong, or can't place, tell you exactly what capability your users want and you don't have. **Your misroutes are your roadmap.**"

17.3 Lab (15 min)

```
Task:  List 20 real questions your users would ask.
       Sort each into: documents / database / live system / can't answer.

What you'll find: usually 20–40% of questions are NOT document
questions. Most bots silently answer these wrongly.
```

Module 18 — Agentic: the bot that tries again (75 min)

18.1 The difference (20 min)

> "Everything so far is a straight line. Question in, one search, answer out. **One attempt.** If the search fetched the wrong thing, the answer is wrong.
>
> **Agentic means the bot decides what to do.**
>
> The comparison that works:

> **What you've built is a vending machine.** Press a button, one item drops. Wrong item? Nothing you can do.
>
> **Agentic is a good librarian.** You ask. They go to the shelf, bring something back, *read it*, and think: 'this only answers half — let me check the appendix too.' They go again. Maybe a third time.
>
> **And when it genuinely isn't in the collection, they come back and tell you so — instead of inventing something.**"

18.2 The loop (20 min)

```
THINK      What do I need to find out?
  ↓
LOOK       Search for it
  ↓
READ       What did I get?
  ↓
JUDGE      Is this enough?
  ├── Yes → write the answer with sources
  └── No  → change the search, or try somewhere else
            ↑ back to LOOK
            ⚠️ ALWAYS with a limit — or it runs forever
```

18.3 When it's worth it — and when it isn't (20 min)

> "**Worth it when:**
>
> - The question needs several lookups in sequence
> - The question is vague and needs narrowing down
> - The answer lives across several different systems
> - 'I couldn't find this, here's what I tried' is genuinely a good outcome
>
> **Not worth it when:**
>
> - Simple, high-volume questions where speed matters
>
> **The numbers:** a well-tuned Advanced bot answers an FAQ in under a second for a fraction of a cent. An agent takes twelve seconds and thirty times the tokens **to reach the same answer.**"

18.4 The safety rails (15 min)

> "If you build one of these, **these are not optional.**
>
> ```
> Maximum number of tries           (5 to 10)
> Maximum total cost per question
> Maximum time before giving up
> A required 'I give up' path
> Loop detection — if it searches the same thing twice, stop
> ```
>
> Without these, one badly-worded question can loop for twenty minutes and cost more than your entire month."

Module 19 — Smart routing: the money saver (40 min)

> "Here's the pattern that makes agentic affordable.
>
> **Don't put every question in the expensive vehicle.**
>
> ```
> Simple question   → answer directly, cheaply
> Normal question   → one search, as you built
> Complex question  → the full agent loop
> ```
>
> Numbers on 10,000 questions a day:
>
> ```
> Everything simple:    $4,500/month  — but fails on hard questions
> Everything agentic:  $22,200/month  — nearly 5× more
> Sorted by type:       $9,810/month  — 56% cheaper than all-agentic,
>                                       with agentic quality where it matters
> ```
>
> **That's the whole argument. Sort your questions first.**"

PART 4 — QUESTIONS YOU WILL BE ASKED

Prepare these. They come up in almost every room.

Question	Short answer to give
	"It changes what the job is. What shrinks is the mechanical

"Will this replace my job?"	middle — first drafts, copying data between systems. What grows is deciding what should be built and **checking whether the output is right**. The person who says 'the AI wrote it, it's probably fine' becomes a risk. The person who says 'I checked these three things and one was wrong' becomes more valuable. Verification is the skill that appreciates."
"Is my company data safe?"	"Depends entirely on your setup. If you use a public chat tool, your documents go to that company. If you use a business account, there are usually contractual guarantees. If your data can't leave the building, it can all run on your own servers. **Ask your IT team before uploading anything sensitive.** Today, use the practice documents."
"Can it learn from our corrections?"	"Not by itself, and this surprises people. It doesn't remember between conversations. If you correct it, that correction vanishes. The way to 'teach' it is to **fix the documents** or **add the corrected answer to the library.**"
"Why is it confident when it's wrong?"	"Because confidence isn't a signal it produces. It's always writing the most plausible next words. Plausible-sounding is what it does — whether or not it's right. **This is exactly why citations matter.** Don't trust the tone, check the source."
"How much does it cost to run?"	"Most first bots: $30–100 a month. The two things that blow it up are long conversations and using the most expensive model for everything. We covered both in Module 13."
"Do I need to learn to code?"	"Not to build what you built this week. Not to understand it, or to run a project, or to judge a vendor. You'd need code to build something serving thousands of users — but you'd hire for that, and **you'd now know what to ask for.**"
"Which is the best AI model?"	"They're all close for this kind of work. **The model isn't your bottleneck — your library is.** A mid-tier model with a good library beats the best model with a bad one, every time, and costs a fraction."
"How long to build a real one?"	"A prototype like yours: days. Something you'd put in front of customers: two to four months. And the time goes to **document quality, permissions, and testing** — not the AI part."
"Can it handle Word/Excel/PowerPoint?"	"Yes, all of them. Excel with a caveat — it can look things up, but **it cannot reliably total a column.** That needs a

	database."
"What if two documents disagree?"	"It'll usually pick one, silently — which is a real risk. Good practice: keep only current approved versions in the library, and mark documents with dates so newer ones win."
"Can we use it in other languages?"	"Yes, though non-English text costs two to five times more for the same content, and quality is usually a bit lower. Worth testing with your actual content before committing."

PART 5 — HANDOUTS

Handout 1 — The glossary (print one per student, hand out Day 1)

Word they'll hear	What it actually means
LLM / Model	The AI that writes the answers. A very good autocomplete.
RAG	Fetching the right pages before answering. That's all.
Token	Roughly three-quarters of a word. The unit you're billed in.
Chunk	One index card. A piece of a document.
Chunking	Cutting documents into cards.
Embedding	Turning text into a list of numbers so similar meanings sit close together.
Vector	That list of numbers.
Vector database	The filing cabinet that stores them.
Semantic search	Searching by meaning rather than exact words.
Keyword / BM25 search	Searching by exact words. Like Ctrl+F.
Hybrid search	Doing both at once.

Retrieval	Fetching the cards.
Top K	How many cards to fetch.
Reranking	Reading the fetched cards and picking the best few.
Context window	How much text the AI can hold at once.
Prompt	The instructions you give the AI.
Grounding	Telling it to answer only from the documents.
Hallucination	A confident, made-up answer.
Metadata	The labels on each card — source, page, date, who can see it.
Agent	A bot that decides what to do and can try again.
Evaluation	Your test sheet.

Handout 2 — The settings sheet (hand out Day 3)

```
SETTINGS YOU WILL BE ASKED TO CHOOSE, AND WHAT TO PICK

Chunk size            500 words to start
                      300 if your documents are dense and technical
                      800 if they're flowing prose

Overlap               10–15%

Chunking method       By heading/section — NOT fixed length

Add context to chunks ON. Always. Biggest cheap improvement.

Search type           HYBRID  (meaning + exact words)

Reranking             ON

Fetch (top K)         50 if reranking is on, then keep 5
                      5–8 if reranking is off

Model                 Mid-tier for question-answering
                      Top-tier only for complex analysis

Temperature           0 to 0.2   (factual, consistent)
```

Chat history	Trim or summarise after ~10 messages
Instruction	Must include: "only from these documents", "cite the source", "say if you can't find it"

Handout 3 — The fault-finding card (hand out Day 4)

```
MY BOT GAVE A WRONG ANSWER. WHAT DO I DO?

STEP 1 — Look at what it fetched.
         (Your tool shows this. Find it. Use it.)

Was the right information in there?

   NO  →  It's a LIBRARY problem.
          ├─ Is the document even in the library?
          ├─ Did it read the file properly? (check the extracted text)
          ├─ Are the cards cut badly? (read 5 of them)
          ├─ Is hybrid search on?
          └─ Is reranking on?

   YES →  It's an ANSWERING problem.
          ├─ Are you sending too many cards? (try fewer)
          ├─ Is the grounding instruction there?
          ├─ Are two documents contradicting each other?
          └─ Is the card itself wrong or out of date?

⚠️  MOST PEOPLE START BY REWRITING THE INSTRUCTIONS.
    MOST PROBLEMS ARE IN THE LIBRARY.
```

PART 6 — TRAINER'S DELIVERY NOTES

6.1 Timing summary

DAY 1 — UNDERSTANDING		4h 00m
M1	Hour One Win (hands-on)	30m
M2	How the AI works	60m
M3	What RAG is and isn't	60m
M4	Choosing your documents + LAB 1	60m

```
  Breaks                                              30m

DAY 2 — BUILDING THE LIBRARY                        4h 00m
  M5   Reading documents properly                     60m
  M6   Cutting into cards                             75m
  M7   Filing the cards + LAB 2                       75m
  Breaks                                              30m

DAY 3 — FINDING THE RIGHT PAGE                      4h 00m
  M8   Understanding the question                     60m
  M9   Two searches + reranking                       75m
  M10  LAB 3 — tuning                                 45m
  Breaks                                              60m

DAY 4 — ANSWERING AND LAUNCHING                     4h 00m
  M11  Instructing the assistant                      60m
  M12  Testing                                        60m
  M13  Cost                                           45m
  M14  LAB 4 — finish                                 60m
  M15  Closing                                        15m
  (runs 20m over — trim M13 or extend the day)

DAY 5 — OPTIONAL ADVANCED                           4h 00m
  M16  The four levels                                45m
  M17  Hybrid and routing                             60m
  M18  Agentic                                        75m
  M19  Smart routing                                  40m
  Breaks                                              20m

TAUGHT TOTAL (Days 1-4)                             16h
HOMEWORK                                             6h
──────────────────────────────────────────────────────────
TOTAL                                               22h
(+4h if Day 5 is included)
```

6.2 If you're running short on time

Cut in this order:

```
1st to cut   M13 cost detail — hand out the numbers instead
2nd          M8.2 query rewriting theory — demo it, don't explain it
3rd          M5.2 tables and pictures — mention, don't demonstrate
4th          M2.4 open Q&A — take questions by email

NEVER cut:   LAB 2 (they get a working bot)
```

```
LAB 3 (they see their own improvement)
M11.1 (the grounding instruction)
M12  (the test sheet)
```

6.3 The four moments that make this course work

> **1. The Hour One Win (M1).** They see it work before they learn anything. Never start with theory.
>
> **2. Reading their own chunks (LAB 2, step 6).** The moment chunking stops being abstract. They will skip this. Make them do it.
>
> **3. The before/after score (LAB 3).** They watch their own number go up because of a decision they made. This is where the course clicks.
>
> **4. The Day 1 flipchart revisited (M15.1).** Every frustration from Monday gets a name and a fix. Most satisfying moment of the week.

6.4 Common trainer mistakes with this audience

Mistake	Do instead
Starting with how transformers work	Start with a working bot. Mechanism comes later, or never.
Using more than one analogy	One library, all four days. Point at the wall poster.
Saying "just" ("just turn on reranking")	Nothing is "just" anything to a beginner
Letting setup eat the first hour	Accounts created a week before, by email
Answering a question with more jargon	If they didn't understand the word, a second word won't help
Skipping the failures	The things that don't work are the curriculum
Letting them change three settings at once	One change, re-test, record. Enforce it.
Broad use cases	Push everyone to something narrow. "HR leave questions", not "everything"

6.5 Success measures for the course itself

```
By end of Day 2:  100% of students have a bot answering from
                  their own documents
By end of Day 3:  90% have improved their score measurably
By end of Day 4:  100% have a test sheet and a cost estimate
Two weeks later:  At least 30% have shown it to someone at work

If the Day 2 number isn't 100%, the problem is almost always
setup, not teaching. Fix setup before you change the content.
```

End of course plan. Tool names verified September 2026 — check current versions before delivery.